

MANUAL
PEDAGÓGICO • V1.0
• 15 SEPT 2026

PalmerP

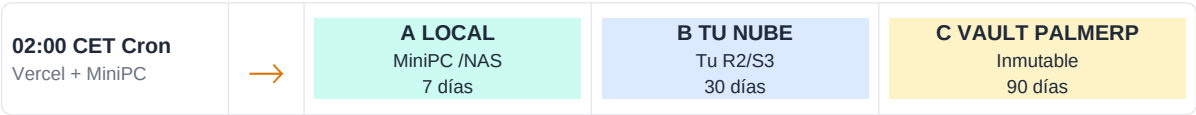
Triple Backup 3-2-1

Instrucciones sencillas, claras y bonitas
para dormir tranquilo: tus datos a salvo aunque falle Vercel o Supabase

3	2	1	02:00
copias cada noche	medios distintos	copia off-site inmutable 90 días	hora fija Europe/Madrid

■ La promesa en una frase

Cada noche a las **02:00** se guardan **2 artefactos** (*logical JSON + pg_dump físico*) cifrados **solo con tu clave** en **3 lugares**: **A)** disco del MiniPC del cliente, **B)** tu R2/S3 propio, **C)** vault central PalmerP inmutable 90 días. PalmerP no puede abrir el candado sin ti.



Para quién es este manual: dueño del negocio + persona que instala el MiniPC + quien gestiona Vercel/Supabase. Sin tecnicismos. Sigue los pasos en orden, marca cada casilla ■.

■■ Cloudflare R2 — 10 GB gratis

Tus backups B y C usan **Cloudflare R2** (`src/lib/storage.ts:3` + `storage-provider.ts:11` con `https://{R2_ACCOUNT_ID}.r2.cloudflarestorage.com`). R2 da **10 GB gratis + 0 € por descarga** (vs AWS S3 que cobra). 2-10 MB/noche × 90 días = 0,2-0,9 GB → gratis. Necesitas 1 cuenta Cloudflare para el cliente (bucket **cliente-backups**) y 1 para tu vault (**palmerp-vault**). Pasos en cap. 2b.

Índice

1	Qué guarda y dónde (en 1 minuto)	3
2	Antes de empezar: lo que necesitas	3
2b	Cloudflare R2 — 10 GB gratis (paso a paso)	3
3	Instalación paso a paso (15 min)	4
4	El MiniPC: agente local 02:00	6
5	Comprobar cada mañana (1 min)	7
6	Restaurar: cuando algo sale mal	8
7	Fleet: cómo gestionas tú las instancias	9
8	Preguntas frecuentes	10

1 Qué guarda y dónde (en 1 minuto)

Cada noche se crean **2 artefactos** por cliente:

- **Logical JSON** — foto limpia de tus datos (usuarios, contactos, tiendas, pedidos, auditoría) filtrada por tu empresa. Sirve para traer usuarios/contactos sin borrar nada. Lo crea Vercel y el MiniPC.
- **Physical pg_dump** — clon exacto de la base de datos (tablas, claves, secuencias). Solo lo crea el MiniPC porque Vercel no tiene *pg_dump*. Es el que te salva si hay que reconstruir todo.

Destino	Dónde vive	Retención	Si falla Vercel
A LOCAL	Disco <code>/data/backups/palmerp</code> del MiniPC siempre ON	7 días 4 semanales aparte	■ Sigue guardando
B TU NUBE	Tu propio bucket R2/S3 (tu cuenta)	30 días	■ Sigue guardando (vía MiniPC)
C VAULT	Vault central PalmerP (<code>vault/tenants/...</code>) inmutable	90 días nunca borra el cliente	■ Sigue guardando

■ Clave solo tuya

El cifrado usa **solo tu BACKUP_ENCRYPTION_KEY**. PalmerP guarda un **.enc opaco** y **no puede abrirlo sin ti**. Si pierdes la clave, el backup es irre recuperable. Guarda la clave impresa (QR) en caja fuerte + en el NAS cifrado.

■ Tranquilidad

Aunque Vercel y Supabase caigan, tienes 2 copias fuera de ellos (MiniPC + tu nube). El vault es el tercer seguro.

2 Antes de empezar: lo que necesitas

Reúne esto en 5 minutos. Marca cada casilla:

- Cuenta **Supabase** del cliente (URL pooler `6543 ?pgbouncer=true`) y **DATABASE_URL**
- Cuenta **Vercel** del cliente (proyecto importado desde GitHub fork)
- Cuenta **R2/S3** del cliente para destino B (ej. Cloudflare R2: Account ID + Access Key)
- Cuenta **R2** tuya para Vault C (separada, bucket **palmerp-vault**)
- MiniPC del cliente siempre encendido con **Ubuntu/Debian + Node 20 + pg_dump**
- Una clave de 32 bytes: genera con **openssl rand -base64 32** (44 caracteres)

■ Genera la clave (una vez)

En tu terminal:

```
openssl rand -base64 32 | tr -d '\n'
```

Copia el resultado. Es tu **BACKUP_ENCRYPTION_KEY**. No la pongas en Git. Imprímela en QR y guárdala.

2b Cloudflare R2 — 10 GB gratis, 0 € por descarga

Tus backups B y C usan **Cloudflare R2** (`src/lib/storage.ts:3` + `storage-provider.ts:11` con `https://{R2_ACCOUNT_ID}.r2.cloudflarestorage.com`). R2 da **10 GB gratis + 10M operaciones/mes + 0 € egress** (AWS S3 cobra). $2\text{-}10\text{ MB/noche} \times 90\text{ días} = 0,2\text{-}0,9\text{ GB} \rightarrow$ gratis. Necesitas 2 buckets: **cliente-backups** (B, en cuenta del cliente, 30d) y **palmerp-vault** (C, en tu cuenta, 90d inmutable).

Paso 1 — Cuenta Cloudflare (2 min)

Ve a <https://dash.cloudflare.com/sign-up> → email + password → verifica. Plan **Free 0 €**. Copia tu **Account ID** (arriba derecha, URL `.../accounts/<ACCOUNT_ID>`). Para R2 te pedirá añadir tarjeta pero no cobra los 10 GB.

Paso 2 — Activa R2 y crea buckets (2 min)

Menú **R2 Object Storage** → **Enable R2** → **Create bucket** dos veces:

- Cliente (B): Name **cliente-backups**, Location Automatic → Create
- Vault (C) en tu cuenta: Name **palmerp-vault**, Location Automatic

En cada bucket → Settings → **Public access: Not public** (backups nunca públicos).

Paso 3 — API Tokens R2 (2 min por cuenta)

R2 → **Manage R2 API Tokens** → **Create API Token** → Name **palmerp-backup-cliente-x** → Permissions **Admin** (o Object Read & Write) → TTL Forever → Create → copia **Access Key ID** (32 chars) y **Secret Access Key** (64 hex) — **no se vuelven a mostrar**.

Guarda: **R2_ACCOUNT_ID** (Account ID) + **R2_ACCESS_KEY_ID** + **R2_SECRET_ACCESS_KEY** + **R2_BUCKET_NAME**. Repite para vault.

Paso 4 — Rellena .env y prueba

En **Vercel Env + MiniPC .env**:

```
CLIENT_BACKUP_S3_ENDPOINT="https://<ACCOUNT_ID_CLIENTE>.r2.cloudflarestorage.com"
```

```
CLIENT_BACKUP_S3_BUCKET="cliente-backups"
```

```
PALMERP_VAULT_R2_ACCOUNT_ID="<ACCOUNT_ID_TUYO>"
```

```
PALMERP_VAULT_R2_BUCKET_NAME="palmerp-vault"
```

Prueba: **npm run backup:agent** debe decir **[CLIENT_STORAGE] uploaded** y **[VAULT] uploaded** (no *not configured*). R2 → Overview verás **Storage: 10 GB / 10 GB Free**.

■ Gratis de verdad

Tus .enc son céntimos en R2. Si un millón descarga un módulo, tu factura egress es 0 € — por eso [hosting_and_business_strategy.md:113](#) recomienda R2.

3 Instalación paso a paso (15 min)

Sigue en orden. Si un paso falla, no avances.

1

Clona el núcleo desde GitHub

Crea el proyecto del cliente a partir del repo base **palmerp** (template). Así Vercel detectará cada **push a main** y desplegará solo.

```
$ git clone https://github.com/tu/palmerp.git cliente-x && cd cliente-x
```

2

Configura .env.local y Vercel Env

Copia **.env.example** → **.env.local** y rellena. Luego replica las mismas vars en **Vercel** → **Settings** → **Environment Variables** (Production).

```
$ cp .env.example .env.local # edita DATABASE_URL, NEXTAUTH_SECRET, BACKUP_ENCRYPTION_KEY, ...
```

Variable	Ejemplo	Dónde
DATABASE_URL	postgresql://...@pooler.supabase.com:6543/postgres?pgbouncer=true	Supabase cliente
NEXTAUTH_URL	https://cliente.vercel.app	Vercel dominio
NEXTAUTH_SECRET	openssl rand -base64 32	Vercel
BACKUP_ENCRYPTION_KEY	44 chars base64 (tu clave)	Vercel + MiniPC
BACKUP_LOCAL_DIR	/data/backups/palmerp	MiniPC
CLIENT_BACKUP_S3_*	Endpoint + Bucket + Keys R2 cliente	Vercel + MiniPC
PALMERP_VAULT_R2_*	Vault PalmerP (tu R2)	Vercel + MiniPC
CRON_SECRET / FLEET_API_KEY	Bearer token aleatorio	Vercel + MiniPC

■ No subas .env a Git

El **.gitignore** ya ignora **.env***. Verifica con **git status** que no aparece. Las claves solo viven en Vercel Env y en el MiniPC.

3

Inicializa base de datos

Crea tablas y usuario admin del tenant.

```
$ npx prisma migrate deploy && npm run instance:provision -- --slug cliente-x --name "Cliente X SL" --admin-email admin@cliente.es
```

4

Conecta Vercel a GitHub

En Vercel → **Add New Project** → **Import from GitHub** selecciona el fork del cliente. Production Branch = **main**. Cada **git push origin main** desplegará auto.

```
$ git push origin main # verifica en Vercel → Deployments que sale verde
```

5

Verifica el cron 02:30

Vercel ya tiene **vercel.json** → **crons: 30 2 * * ***. Comprueba en Vercel → Settings → Cron Jobs que aparece **/api/cron/daily-backups**. No toques la hora.

6

Prueba un backup manual

Lanza un backup sin esperar a las 02:00. Debe devolver **success/partial** con 3 destinos.

```
$ curl -H "Authorization: Bearer $CRON_SECRET" https://cliente.vercel.app/api/cron/daily-backups
```

■ ¿Todo verde?

Si el JSON trae **"overallStatus": "success"** y ves 3 destinos OK, ya duermes tranquilo. Si sale **"not configured"** en algún destino, revisa ese bloque de vars.

4 El MiniPC: agente local 02:00

El cerebro local + el guardián del backup físico. Siempre encendido.

Por qué hace falta

Vercel no tiene `pg_dump`, así que solo puede guardar el JSON. El MiniPC sí crea el `.physical.sql.gz.enc` (clon exacto) y además es tu copia A local que no depende de internet. Si Vercel cae a las 02:00, el MiniPC igualmente guarda en los 3 sitios.

Instalación en 5 minutos

1

Prepara el MiniPC

Ubuntu/Debian, Node 20+, `pg_dump`, disco con espacio.

```
$ sudo apt update && sudo apt install -y nodejs postgresql-client && node -v && pg_dump --version
```

2

Clona y configura

Mismo repo + mismo `.env` que Vercel (copia las vars).

```
$ git clone https://github.com/tu/palmerp.git /opt/palmerp && cd /opt/palmerp && npm ci --legacy-peer-deps && cp .env.example .env
```

3

Prueba manual

Debe crear `/data/backups/palmerp/tenants/.../daily/*.enc` y subir a B y C.

```
$ npm run backup:agent # o node scripts/backup-agent.mjs --once
```

4

Automatiza a las 02:00

Systemd timer es más fiable que cron.

```
$ sudo cp scripts/systemd/palmerp-backup.* /etc/systemd/system/ # ver nota abajo sudo systemctl daemon-reload && sudo systemctl enable --now palmerp-backup.timer systemctl list-timers | grep palmerp
```

■ Systemd si no tienes los archivos

Crea tú `/etc/systemd/system/palmerp-backup.service` con `ExecStart=/usr/bin/node /opt/palmerp/scripts/backup-agent.mjs --once` y `palmerp-backup.timer` con `OnCalendar=02:00`. El script ya está en `scripts/backup-agent.mjs:1`.

Qué hace cada noche

1) Lee tu Supabase directo (no Vercel) → 2) gzip → 3) cifra solo con tu clave → 4) sha256 → 5) escribe **LOCAL** + sube **CLIENT_STORAGE** + **VAULT** → 6) escribe **BackupLog** + **AuditLog** **BACKUP_HEARTBEAT** → 7) borra lo de 7/30 días (vault nunca).

5 Comprobar cada mañana (1 minuto)

No necesitas abrir 3 nubes. Mira un solo semáforo.

Color	Significa	Qué hacer
● Verde	Backup <24h OK en 3 destinos	Nada. ■
● Amarillo	Backup 24-48h o solo 2 destinos	Revisa vars del destino fallido, lanza backup manual.
● Rojo	Backup >48h o fallido	Actúa hoy: revisa MiniPC encendido, credenciales R2, logs.

Dónde mirar

Opción A - Superadmin (tú): **GET /api/fleet/backup-heartbeat** → JSON con **status[] {slug, lastBackupAt, ageHours, color}**. Monta un dashboard sencillo con esos colores.

Opción B - Cliente: **GET /api/admin/restore?tenantId=...** lista últimos **BackupLog** o entra por SSH al MiniPC y mira **ls -lh /data/backups/palmerp**.

Comando rápido

```
curl -H "Authorization: Bearer $FLEET_API_KEY" https://control.palmerp.es/api/fleet/backup-heartbeat | jq .status
```

Log diario

Cada backup escribe **BackupLog** (tenantId, destination, fileKey, checksum, size, encrypted, status) y **AuditLog BACKUP_RUN / BACKUP_HEARTBEAT**. Si ves **PARTIAL** es que algún destino falló pero al menos uno guardó.

6 Restaurar: cuando algo sale mal

■ ■ Antes de restaurar

Para y respira: **no sobrescribas sin dryRun**. Prueba primero en local. El restore lógico **nunca borra**, solo añade usuarios/contactos que faltan. El físico sí reconstruye todo.

Caso 1 - Recuperar un usuario/contacto borrado (lógico)

1

Lista backups

Elige el **.logical.json.gz.enc** de ayer.

```
$ curl "https://cliente.vercel.app/api/admin/restore?tenantId=xxx" | jq .logs[0]
```

2

Descarga el .enc

Bájalo de donde prefieras: **MiniPC /data/backups**, tu **R2 cliente**, o **Vault vault/tenants/...** Cópialo a tu portátil.

```
$ scp mini:/data/backups/palmerp/tenants/xxx/daily/2026-09-14.logical.json.gz.enc ./
```

3

Verifica (dryRun)

Descifra con tu clave y valida sin tocar base de datos.

```
$ curl -X POST https://cliente.vercel.app/api/admin/restore -H "Content-Type: application/json" -d
'{"tenantId":"xxx","fileKey":"tenants/xxx/daily/...logical.json.gz.enc","encBase64":"...base64...","dryRun":true}' | jq .preview
```

4

Restaura

Mismo POST con **dryRun:false** → crea usuarios/contactos faltantes.

```
$ dryRun:false # revisa respuesta {restoredUsers}
```

Caso 2 - Reconstruir toda la base (físico)

Solo con el artefacto **.physical.sql.gz.enc** del MiniPC/Vault. Pasos en tu portátil con la clave:

1) Descifra (usa **src/lib/backup/crypto.ts:decryptBuffer** o script) → 2) **gunzip** → 3) **psql "\$DATABASE_URL" < dump.sql**

Nunca lo hagas en producción sin parar la app y avisar al cliente. Hazlo primero en staging.

Desencriptado manual (si no quieres API)

El formato es **iv 12B | tag 16B | ciphertext** AES-GCM. Usa el helper **decryptBuffer** de **src/lib/backup/crypto.ts:1** o el snippet de **scripts/backup-agent.mjs:1** (función **resolveKey**). Sin la clave da error **auth failed** — es normal.

■ Physical requiere MiniPC

Si solo ves **logical** en Vercel, es normal: Vercel no genera físico. Busca el **physical** en el MiniPC o en el Vault tras la noche que el agente corrió.

7 Fleet: cómo gestionas tú las instancias

Cada cliente tiene su Vercel + Supabase + GitHub + MiniPC. Tú no tocas su base de datos, pero ves salud.

Modelo Fleet independiente

Repo base **palmerp** es **template**. Cliente hace **fork/clone** → importa en su Vercel. Tu repo base es **upstream: git remote add upstream https://github.com/tu/palmerp.git** → **git fetch upstream && git merge upstream/main** → **git push origin main** → Vercel del cliente despliega solo. Cada modo nuevo se instala desde repo base vía **POST /api/admin/modes/install** (manifiesto **registry.json**).

Tú controlas	Cliente controla
Vault central inmutable 90d Heartbeat/semáforo fleet	Su R2/S3, su Supabase, su MiniPC, su clave
Repo base + manifiesto modos	Su fork + su Vercel deploy auto
Soporte + updates upstream	Coste Vercel/Supabase lo paga él directo

Gestión sin invadir

Cada instancia hace **POST /api/fleet/backup-heartbeat** cada noche (agente o cron). Tú montas un dashboard que consume **GET /api/fleet/backup-heartbeat** y pinta el semáforo. Si quieres acceso delegado, que te añada como **Member** en su Vercel Team / Supabase Org (no necesitas su DATABASE_URL para ver salud).

■ Costes claros

El cliente paga su Vercel + Supabase + R2 directo. Tú pagas solo el Vault central (un bucket R2). Margen alto, sin sorpresas.

8 Preguntas frecuentes

¿Y si pierdo la BACKUP_ENCRYPTION_KEY?

No hay vuelta atrás: el .enc no se abre. Por eso la generas una vez, la copias en **Vercel Env + MiniPC .env + papel QR en caja fuerte + NAS cifrado**. Haz copia ya.

¿Por qué dos artefactos?

El **logical** es ligero y sirve para recuperar usuarios/contactos sin parar nada. El **physical** es el salvavidas total. Con ambos duermes tranquilo.

¿Puedo cambiar la hora de 02:00?

Sí, edita **vercel.json** → **crons 30 2 * * *** y el **systemd timer OnCalendar** del MiniPC. Mantén ambos iguales.

¿Cuánto ocupa?

JSON gzip ~90% menos. Un negocio medio: logical 2-10 MB cifrado, physical 10-100 MB. En R2 90 días son céntimos.

¿Y si el MiniPC se apaga?

Has confirmado que estará siempre ON. Si se apaga, esa noche solo tendrás logical en Vercel (amarillo). Al encender, el agente retoma a las 02:00 siguiente.

¿Cómo sé que el backup es bueno?

El runner hace **gzip** → **encrypt** → **sha256** y guarda **BackupLog.checksum**. El restore dryRun descifra y valida JSON. El Vault es inmutable, nadie lo borra.

¿PalmerP puede ver mis datos?

No. El vault guarda **.enc opaco**. Sin tu clave no hay descifrado. Ni siquiera con acceso al bucket.

Checklist final: pégalo en la pared ■

- **Clave** generada y guardada en 4 sitios (Vercel, MiniPC, papel QR, NAS)
- **Vercel cron 02:30** visible en dashboard + **systemd timer 02:00** activo
- **Backup manual** probado: 3 destinos success, 2 artefactos (logical siempre, physical en MiniPC)
- **Semáforo** verde <24h en **/api/fleet/backup-heartbeat**
- **Restore dryRun** probado con un .enc real
- **Vault** con lifecycle 90d inmutable, **CLIENT_STORAGE** 30d, **LOCAL** 7d

¿Dudas? Escríbenos. Si este manual te ha dado tranquilidad, ha cumplido su misión. ■

PalmerP • Hecho con mimo en Europa • 2026